Availability

the nines, MTBF vs MTTR, series vs parallel

Watch the 3-min video

In one line: availability is the share of time a system serves requests successfully; it depends on how often things fail and how fast you recover, and it multiplies down in series and up with redundancy.

1 The nines

Availability	Name	Downtime per year	Per month
99%	two nines	~3.65 days	~7.3 h
99.9%	three nines	~8.76 h	~43.8 min
99.99%	four nines	~52.6 min	~4.4 min
99.999%	five nines	~5.3 min	~26 s

2 MTBF and MTTR

Two numbers drive availability: **MTBF** (mean time between failures: how often things break) and **MTTR** (mean time to recovery: how fast you fix them).

```
availability = MTBF / (MTBF + MTTR)
```

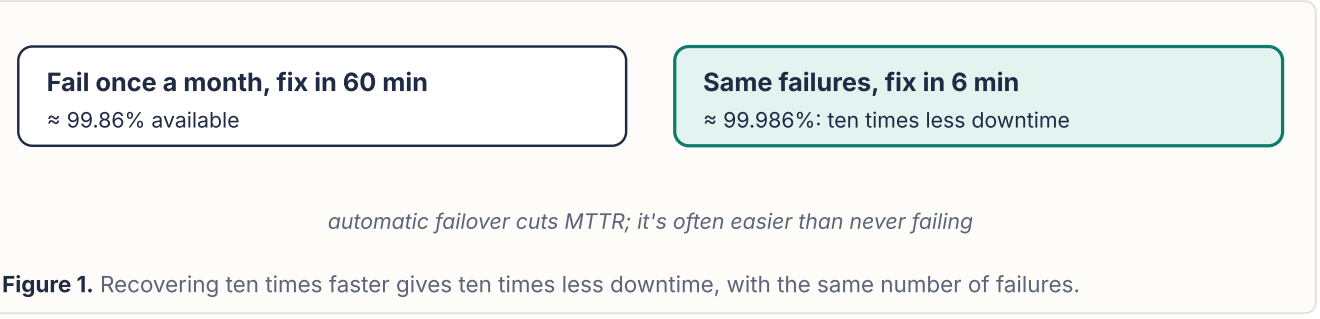


Figure 1. Recovering ten times faster gives ten times less downtime, with the same number of failures.

3 Series and parallel

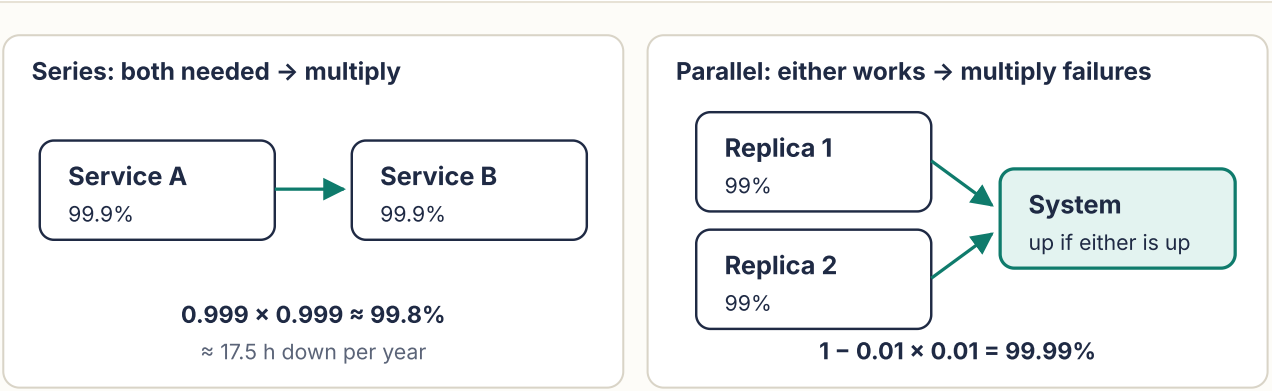


Figure 2. Dependencies in a chain multiply availability down; independent replicas multiply the failure chance down.

- **Series** (a request needs A and B): multiply the availabilities. Every extra dependency on the critical path lowers the total.
- **Parallel** (either replica can serve): multiply the *failure* chances. Two 99% replicas fail together only 0.01% of the time.
- The parallel maths holds only if failures are **independent**. Two replicas in the same rack, on the same power or with the same bad config fail together.

WORKED EXAMPLE

A checkout calls the API (99.95%), the payment service (99.9%) and the database (99.95%) in series: 0.9995 × 0.999 × 0.9995 ≈ **99.8%**, about 17.5 hours down a year. Run the database as a primary with a standby in another zone, each 99.95%: the pair is $1 - 0.0005^2 \approx 99.99998\%$, and the chain improves to ≈ **99.85%**. Now the payment service is the weakest link.

4 How to improve availability

- **Redundancy** across zones or regions, with failures as independent as you can make them.
- **Remove single points of failure** on the request path.
- **Detect fast, fail over automatically**: health checks and automatic promotion cut MTTR from an hour to seconds or minutes.
- **Timeouts and careful retries**, so a slow dependency doesn't hang everything.
- **Shorten the critical path**: make optional calls asynchronous or degrade gracefully when they fail.

COMMON MISTAKES

- Adding dependencies without counting the multiplied availability.
- Assuming replicas fail independently when they share a zone or config.
- Focusing only on MTBF and ignoring recovery time.
- Measuring uptime from a health check while real requests fail.

5 Key terms

Availability: share of time serving successfully

MTTR: mean time to recovery

Parallel: any part enough: multiply failures

MTBF: mean time between failures

Series: all parts needed: multiply availability

Critical path: calls a request can't succeed without

6 Interview questions

Q1. Two services at 99.9% each, in series. What's the total? BASIC
About 99.8%, because you multiply: 0.999 × 0.999 ≈ 0.998, roughly 17.5 hours of downtime a year.

Q2. What do MTBF and MTTR mean, and how do they give availability? BASIC
MTBF is how long between failures; MTTR is how long to recover. Availability = MTBF / (MTBF + MTTR).

Q3. Two independent replicas at 99% each. What's the availability? BASIC
1 − (0.01 × 0.01) = 99.99%, because the system is down only when both fail at once.

Q4. You need 99.99% but each server gives 99.5%. What do you do? SCENARIO
Run several replicas across zones behind a load balancer with fast health checks: two independent 99.5% replicas give $1 - 0.005^2 \approx 99.9975\%$. Then check the rest of the chain (load balancer, database) also meets the target.

Q5. A service fails once a month and takes an hour to fix. Fastest way to improve it? SCENARIO
Cut MTTR: automatic failover and good alerts can bring recovery to ~6 minutes, which takes availability from ~99.86% to ~99.986% without changing how often it fails.

Q6. Your page calls 8 services in series. How do you raise its availability? SCENARIO
Remove non-essential calls from the critical path (make them async or optional with fallbacks), add redundancy to the weakest services, and use timeouts so a slow dependency degrades the page instead of failing it.

Q7. Why might two replicas give much less than the calculated 99.99%? TRICKY
Failures aren't independent: same zone, same power, same bad deploy or config, or a shared dependency like DNS. Correlated failures break the parallel maths.

Q8. Your dashboard shows 100% uptime but users report errors. How? TRICKY
The health check measures “process is up”, not “requests succeed”. Measure availability as the success rate of real requests (an SLI), not a ping.

Next: SD-A-011 · Reliability vs Availability vs Durability backendroadmap.com